Perpetual Contracts as a General Insurance Mechanism

Rohan Badade

rohan@temporal.exchange

March 25, 2026

Abstract

We propose a general framework for implementing insurance contracts as perpetual derivative instruments. The framework consists of two layers: a singleton *apportionment layer* acting as primary insurer, maintaining a registry of insured entities, collecting premiums, and routing payouts; and a pooled perpetual acting as the *reinsurance layer*, whose LP shorts provide aggregate capital. The perpetual operates in one of two oracle states. In the normal state the oracle is zero and longs pay shorts continuously, constituting the insurance premium. In the event state the oracle jumps to a calibrated payout fraction; funding inverts and streams from shorts to longs at the maximum rate for N^* intervals, constituting the payout. Premium and payout are fully decoupled. No reserve, claims committee, or governance vote is required. We establish eight main results covering actuarial fairness, moral hazard elimination, coverage self-enforcement, leverage solvency, payout sufficiency, oracle calibration correctness, singleton necessity, and snapshot stability. We present HIP-3 deployer slashing risk on Hyperliquid as the canonical instantiation.

Contents

1 Introduction **3**
- 1.1 Canonical Instantiation . . . 3
- 1.2 Contributions . . . 3

2 Two-Layer Architecture **4**

3 The Apportionment Layer **5**
- 3.1 Registry . . . 5
- 3.2 Premium Collection . . . 5
- 3.3 Event Trigger and Oracle Calibration . . . 6

3.4 Payout Routing 6

4 Two Oracle States and the Funding Mechanism **8**
4.1 Normal State 8
4.2 Event State 8
4.3 Event Queue 9

5 Insured's Required Position and Coverage Validity **9**

6 Maximum Leverage Cap **10**

7 Post-Event Price Dynamics **10**

8 Main Theoretical Results **11**

9 Deployment Constraints and Security Requirements **12**
9.1 $f_{\max}$ Immutability 12
9.2 Oracle Controller Security 13
9.3 LP Window Persistence 13
9.4 Lambda Observability (HIP-3 Specific) 13

10 Cross-Subsidy, Adverse Selection, and LP Incentives **13**
10.1 Cross-Subsidy in Pooled Coverage 13
10.2 Adverse Selection Dynamics 14
10.3 LP Incentive Model 14

11 Notation Summary **16**

12 Conclusion **17**

References **17**

A Lean 4 Formalisation Record **17**
A.1 Proof Dependency Table 18
A.2 Axiom Register 18
A.3 Pass History and Superseded Theorems 19

1 Introduction

Traditional insurance suffers from three structural inefficiencies: premiums are set by actuarial committees rather than discovered by markets; coverage is episodic, creating renewal friction and gap risk; and claims require human adjudication subject to dispute and delay. For risks that are on-chain observable and parametrically verifiable, none of these features is necessary.

Existing decentralised insurance instruments have addressed individual inefficiencies in isolation. Nexus Mutual replaces the claims committee with a governance vote. Chainproof replaces the premium schedule with a yield benchmark. Parametric depeg products eliminate claims adjudication for a narrow trigger class. None has unified premium discovery, coverage validity, and payout delivery into a single self-contained mechanism.

We show that a two-layer perpetual contract architecture achieves this unification. The outer layer is a singleton smart contract, the *apportionment layer*, which maintains individual policy relationships exactly as a primary insurer does. The inner layer is a pooled perpetual, the *reinsurance layer*, whose funding mechanism delivers aggregate payouts. The apportionment layer is the only entity that interacts with the perpetual; individual insured entities interact only with the apportionment layer.

1.1 Canonical Instantiation

Throughout we instantiate the framework on Hyperliquid's HIP-3 protocol. Under HIP-3, any entity staking 500,000 HYPE may deploy a permissionless perpetual futures market on HyperCore. Validators may slash up to 100% of the stake for malicious or negligent oracle operation. The slashing event is on-chain observable; λ_i is parametrically readable from the chain at T^*, subject to the engineering requirements described in §9.4. The at-risk capital V_i is the deployer's stake in HYPE tokens, fixed at registration.

1.2 Contributions

1. A two-layer architecture separating individual policy management from aggregate risk capacity.

2. A two-oracle-state mechanism in which premium and payout are fully decoupled, implemented purely through the perpetual's funding mechanics.

3. Eight main theorems and supporting results establishing correctness of the mechanism (see Annexure A for the full formalisation record).

4. Explicit characterisation of deployment security requirements: $f_{\max}$ immutability, oracle controller security, and LP window persistence, each supported by a constructive counterexample showing what breaks if the requirement fails.

2 Two-Layer Architecture

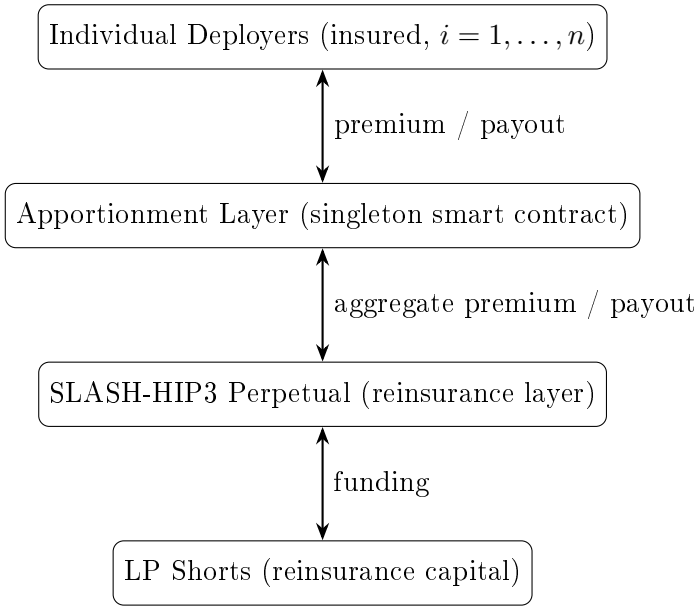

Figure 1: Two-layer architecture. Individual deployers interact only with the apportionment layer; the layer is the sole counterparty of the perpetual.

Apportionment layer (primary insurer). A singleton smart contract on HyperEVM. It is the unique entity permitted to hold a long position on the reinsurance perpetual and the unique entity permitted to update the perpetual's oracle. Its logic — registry, premium collection, event trigger, oracle calibration, and payout routing — is described in full in §3.

Reinsurance perpetual. A single pooled perpetual. It sees only the apportionment layer as its counterparty; individual deployer identities are opaque to it. LP shorts provide the reinsurance capital that absorbs losses during event windows.

Singleton necessity. The singleton constraint on the apportionment layer is a mathematical requirement, not merely an implementation preference. If two independent layers each calibrate using their own partial pool size, the combined payout is $2V_i\lambda_i \neq V_i\lambda_i$ (Theorem 8.9). The result does not rule out multi-layer architectures where layers share a single oracle or use a fixed external V_{snap}; the necessity applies specifically to the calibration identity (22).

Remark 2.1 (Insurance vs. speculation). An entity holding a long position on the reinsurance perpetual *directly* — bypassing the apportionment layer — is a speculator, not an insured. They pay funding continuously (correct direction) but at an arbitrary

notional unrelated to their stake; they receive funding during any deployer's event window (not only their own); and the payout amount is determined by their notional, not by their actual loss. The apportionment layer converts the speculative instrument into proper insurance by enforcing three properties the perp cannot provide alone: the premium is sized to stake ($V_j \cdot P(t)$), the payout is triggered only on the holder's own slashing event, and the payout equals the actual loss ($V_j \cdot \lambda_j$) exactly. The singleton constraint reinforces this separation: if a deployer could also hold a direct long position, they would receive payouts on other deployers' events, conflating insurance recovery with speculative gain. The perp provides capital and price discovery; the layer provides identity and routing. Neither alone constitutes insurance.

3 The Apportionment Layer

The apportionment layer is a singleton smart contract that functions as the primary insurer in the two-layer architecture. It is the unique entity permitted to hold a long position on the reinsurance perpetual and the unique entity permitted to update the perpetual's oracle. Its logic can be decomposed into four sequential concerns: registry, premium collection, event trigger and oracle calibration, and payout routing.

3.1 Registry

Each insured entity i is registered with three fixed fields:

- $V_i \in \mathbb{R}^+$ — at-risk capital in HYPE tokens, fixed at registration and immutable thereafter. Enforces the moral hazard cap $Q_i \leq V_i$ (Theorem 8.3).
- $\pi_i \in (0, 1)$ — risk parameter, certified off-chain by an independent auditor and stored as a fixed value at registration.
- *Active status* — set false on deregistration. Deregistration is disallowed while an event window is active, reinforcing the frozen snapshot (Definition 3.4).

The layer maintains two pool aggregates updated on every registration or deregistration:

$$V_{\text{pool}} = \sum_j V_j, \qquad \pi V_{\text{pool}} = \sum_j \pi_j V_j, \tag{1}$$

enabling $O(1)$ computation of the capital-weighted pool average $\pi_{\text{pool}} = \pi V_{\text{pool}}/V_{\text{pool}}$ and the risk-adjusted premium weight $w_i = \pi_i V_i / \pi V_{\text{pool}}$ (Definition 3.5).

3.2 Premium Collection

In normal state, each insured calls the layer to deposit premium. The actuarially correct premium per funding interval is:

$$d\Pi_i(t) = V_i \cdot P(t) \cdot \Delta t, \tag{2}$$

where $P(t)$ is the current mark price on the reinsurance perpetual — the market's estimate of π_{pool}. Premiums are deposited freely; the layer records each entity's cumulative deposit for coverage validity monitoring (Definition 5.2) but does not enforce a fixed payment schedule per call.

During an active event window, the layer freezes the premium rate at $P(T^-)$ — the last mark price before the window opened — for all entities including the slashed party (Remark 4.4).

3.3 Event Trigger and Oracle Calibration

When a slashing event for entity i is confirmed, the layer executes the following atomically:

$$V_{\text{snap}} \leftarrow V_{\text{pool}}, \qquad O(T^*) = \frac{V_i \cdot \lambda_i}{V_{\text{snap}}}, \qquad \text{pendingPayout} = V_i \cdot \lambda_i. \tag{3}$$

The snapshot and calibration occur in a single operation, eliminating any window between them during which pool composition could change and produce a stale denominator. No entries, exits, or stake changes are reflected in V_{snap} until the event window closes.

The layer then exposes the current oracle value to the oracle relay:

$$O_{\text{current}}(t) = \begin{cases} 0 & \text{normal state} \\ O(T^*) & \text{event state} \end{cases} \tag{4}$$

The relay reads O_{current} every Δt seconds and pushes it to HyperCore. No manual oracle input is required once the layer is deployed — the layer's state fully determines what the perpetual's oracle receives. This is the mechanism by which the singleton oracle control requirement (§9.2) is exercised.

The event window runs for $N^* = \lceil O(T^*)/f_{\max} \rceil$ intervals (Definition 4.2). The layer does not need to store $f_{\max}$ internally: N^* is computed from $O(T^*)$ alone, and the layer's payout obligation $V_i \cdot \lambda_i$ is fixed at trigger time regardless of how many intervals the perp takes to deliver it.

3.4 Payout Routing

After N^* intervals, the layer transfers pendingPayout $= V_i \cdot \lambda_i$ to the slashed entity's address, clears the event state, and resumes normal premium collection at the live mark price $P(t)$. The ceiling excess — the difference between what the perp delivered ($V_{\text{snap}} \cdot N^* \cdot f_{\max}$) and what the layer paid out ($V_i \cdot \lambda_i$) — accumulates in the layer as a reserve buffer (Remark after Theorem 8.6).

Premium withdrawals by the layer operator are blocked during an active event window, ensuring the buffer is available for payout.

Definition 3.1 (Binary loss event). *$\mathcal{E}$ is a binary random variable. The event time $T^* \in [0, \infty)$ is the first time $\mathcal{E} = 1$. The event is on-chain observable and parametrically verifiable.*

Definition 3.2 (At-risk capital and loss fraction). *For protected entity i, $V_i \in \mathbb{R}^+$ is the at-risk capital in HYPE tokens, fixed at registration. The loss fraction $\lambda_i \in [0, 1]$ is the fraction of V_i lost conditional on $\mathcal{E}$, observable on-chain at T^*.*

Definition 3.3 (Risk parameter).

$$\pi_i = \mathbb{E}[\lambda_i \cdot \mathbf{1}[\mathcal{E}]] \in (0, 1). \tag{5}$$

The probability-weighted expected loss fraction. A pure dimensionless number. All idiosyncratic factors are encoded in π_i; their derivation is extrinsic to the mathematics. In the canonical instantiation, π_i is certified by an independent technical auditor; the certification process is load-bearing for pool stability (see §10.2).

Definition 3.4 (Pool total, pool average, and snapshot). *The pool total at time t is $V_{\text{pool}}(t) = \sum_j V_j$. The capital-weighted pool average risk parameter is:*

$$\pi_{\text{pool}}(t) = \frac{\sum_j \pi_j V_j}{V_{\text{pool}}(t)}. \tag{6}$$

The snapshot V_{snap} is taken from the live pool total at the time the event begins processing *— not the time it is observed on-chain. Once taken, V_{snap} is held frozen for the duration of the event window: no entries, exits, or stake changes are reflected until the window closes. Deregistration of any entity is disallowed while an event window is active, reinforcing the frozen snapshot.*

If the pool grows between observation and processing, the oracle value is smaller (payout is conservative); if the pool shrinks, the oracle is larger. Stale observation-time snapshots would miscalibrate in the direction of the pool's trend.

Definition 3.5 (Premium weight). *The risk-adjusted premium share of entity i is:*

$$w_i = \frac{V_i \cdot \pi_i}{\sum_j V_j \cdot \pi_j}. \tag{7}$$

In a single pooled market the premium rate $P(t)$ converges to π_{pool}, so each entity pays $V_i \cdot P(t)$ regardless of w_i. The weight governs the intended *apportionment of risk burden; deviations from it are the cross-subsidy discussed in §10.*

Definition 3.6 (Funding mechanics). *$f_{\max} > 0$ is the protocol maximum funding rate per interval; $\Delta t > 0$ is the funding interval. All values are HYPE-denominated.*

4 Two Oracle States and the Funding Mechanism

The perpetual's mark price $P(t) \in (0,1)$ is order-book determined. The oracle $O(t)$ is controlled exclusively by the apportionment layer. The funding rate per unit notional per interval is:

$$f(t) = \frac{P(t) - O(t)}{\Delta t}. \tag{8}$$

When $f > 0$ longs pay shorts; when $f < 0$ shorts pay longs.

4.1 Normal State

Definition 4.1 (Normal oracle)**.** $O(t) = 0$ *when no event window is active.*

In normal state $f(t) = P(t)/\Delta t > 0$: longs pay shorts continuously. Deployer i's premium per interval is:

$$d\Pi_i(t) = V_i \cdot P(t) \cdot \Delta t. \tag{9}$$

In market equilibrium $P(t) \approx \pi_i$, so the premium rate equals the market's estimate of the expected loss fraction.

4.2 Event State

When slashing of deployer i is observed at T^* and reaches the front of the queue, the layer reads λ_i from chain and takes V_{snap} from the live pool total at that moment (see §3 for the atomic trigger mechanics).

Definition 4.2 (Event oracle and window)**.**

$$O(T^*) = \frac{V_i \cdot \lambda_i}{V_{\text{snap}}} \in (0,1), \tag{10}$$

$$N^* = \left\lceil \frac{O(T^*)}{f_{\max}} \right\rceil. \tag{11}$$

$O(T^*)$ *is sustained for* N^* *consecutive intervals, then resets to* 0*. The term* two oracle states *refers to the regime — Normal* ($O = 0$) *vs Event* ($O > 0$) *— not to a binary oracle value. Within the event regime,* $O(T^*)$ *is a continuous fraction in* $(0,1)$*: larger for a large deployer with a high loss fraction, smaller for a minor partial slash. The* N^* *ceiling construction absorbs this variation automatically — a larger oracle value produces more intervals, a smaller one fewer, with the total payout* $V_{\text{snap}} \cdot N^* \cdot f_{\max} \geq V_i \cdot \lambda_i$ *in all cases (Theorem 8.6).*

Since $O(T^*) \gg P(T^*)$, funding is pinned at $-f_{\max}$ throughout the event window:

$$f(t) = -f_{\max} \qquad \forall\, t \in [T^*,\, T^* + N^* \cdot \Delta t]. \tag{12}$$

Remark 4.3 (Arbitrage convergence). Since $f(t) = -f_{\max} < 0$, arbitrageurs buy longs to capture negative funding, pushing $P(t) \uparrow O(T^*)$ over the event window. Equilibrium within the window is $P(t) \approx O(T^*)$.

Remark 4.4 (Premium collection during event window). During the event window, $P(t)$ is driven toward $O(T^*) = V_i\lambda_i/V_{\text{snap}}$ by arbitrage — a small event-specific value that reflects the slashing payout fraction, not the underlying risk of the surviving deployer population. Using the live mark as the premium rate during this period would cause all deployers to pay near-zero premiums for N^* intervals, then face a spike when the oracle resets — economically incorrect. The apportionment layer therefore freezes the premium rate for all deployers at $P(T^-)$, the last mark price before the event window opened:

$$d\Pi_j(t) = V_j \cdot P(T^-) \cdot \Delta t \qquad \forall\, j,\ t \in [T^*,\, T^* + N^* \cdot \Delta t]. \tag{13}$$

This includes deployer i (the slashed party), whose coverage remains active until payout is complete. The frozen-rate premiums are small relative to the payout flow $V_{\text{snap}} \cdot f_{\max}$ per interval and accumulate in the layer alongside the ceiling excess from Theorem 8.6.

Remark 4.5 (Full decoupling). The normal and event states are governed by entirely separate equations. What is paid in normal state has no bearing on what is received in event state, and vice versa.

4.3 Event Queue

At most one event window is active at any time. If a second slashing event occurs while a window is active, it is queued and processed only after the current N^* intervals complete. When the queued event begins, V_{snap} is taken fresh from the live pool total at that moment.

5 Insured's Required Position and Coverage Validity

Definition 5.1 (Required notional and moral hazard cap)**.** *Each deployer i must maintain a long position (held via the apportionment layer) of notional:*

$$Q_i = V_i. \tag{14}$$

The layer enforces $Q_i \leq V_i$ by computing $\min(\textit{requested}, V_i)$*. This prevents over-insurance: the payout $V_i \cdot \lambda_i$ cannot exceed the actual loss.*

Definition 5.2 (Margin requirement and coverage validity)**.** *Entity i posts margin $M_i(t) \in \mathbb{R}^+$. The margin requirement is:*

$$M_i(t) \geq \frac{V_i}{L_{\max}}, \tag{15}$$

where $L_{\max}$ is the protocol leverage cap. Coverage validity:

$$\mathbf{1}[\text{covered}]_i(t) = \mathbf{1}\left[M_i(t) \geq \frac{V_i}{L_{\max}}\right]. \tag{16}$$

Coverage lapses automatically when (15) is violated; no governance action required.

6 Maximum Leverage Cap

Definition 6.1 (Leverage cap)**.** *Let $\Delta t_{\text{liq}} > 0$ be the maximum liquidation window and $\Delta P^{\text{liq}}_{\max} = \sup_t |P(t + \Delta t_{\text{liq}}) - P(t)|$. The leverage cap is:*

$$L_{\max} = \left\lfloor \frac{1}{\Delta P^{\text{liq}}_{\max}} \right\rfloor, \tag{17}$$

applied symmetrically to longs and shorts.

7 Post-Event Price Dynamics

The post-event dynamics described in this section are qualitative characterisations; their formal derivation requires a model of informed order flow and is left for future work.

Oracle reset. $O(t) \to 0$. Funding immediately becomes positive; surviving deployers who re-establish $Q_j = V_j$ pay shorts, creating downward pressure on the mark.

Mark repricing. The mark converges toward the new pool-implied risk level:

$$P(T^{*+}) = \gamma \cdot \pi^{+}_{\text{pool}}, \qquad \gamma > 1, \tag{18}$$

where $\pi^{+}_{\text{pool}} = \sum_{j \neq i} \pi_j$ and $\gamma > 1$ reflects that event occurrence reveals $\mathcal{E}$ is realisable and potentially updates beliefs about π_j for all surviving deployers. γ is endogenously determined by order flow and is not bounded by any proved result.

Long-run mean reversion. As deployers improve their risk profiles, $P(t) \to \sum_j \pi_j^{\text{improved}} < \pi^{+}_{\text{pool}}$. Higher premiums post-event create financial incentive to reduce π_i without governance.

8 Main Theoretical Results

Theorem 8.1 (Actuarial fairness)**.** *Conditional on $P(t) = \pi_i$ for all $t \in [0, T]$, the expected total premium equals the expected total loss:*

$$\mathbb{E}\left[V_i \int_0^T P(t)\, dt\right] = V_i \cdot \pi_i \cdot T = \mathbb{E}[V_i \cdot \lambda_i \cdot \mathbf{1}[\mathcal{E}] \cdot T]\,. \tag{19}$$

Moreover, $f(t) = P(t)/\Delta t$ is the unique funding rate achieving actuarial fairness in equilibrium.

Proof. In equilibrium $P(t) = \pi_i$: $\mathbb{E}[\int_0^T V_i P\, dt] = V_i \pi_i T = V_i \mathbb{E}[\lambda_i \mathbf{1}[\mathcal{E}]]T$. Uniqueness: if $\alpha \neq 1$, expected premium $= \alpha V_i \pi_i T \neq V_i \pi_i T$. □

Remark 8.2 (Conditionality of Theorem 8.1). Theorem 8.1 requires the mark price to reach the equilibrium $P(t) = \pi_i$. This in turn requires sufficient pool depth, informed participation from both longs and shorts, and the absence of persistent arbitrage barriers — conditions extrinsic to the on-chain mechanism and not guaranteed by the contract. In a pooled market, equilibrium means $P(t) \approx \sum_i \pi_i V_i / V_{\text{snap}}$ (the capital-weighted average), not individual π_i; individual actuarial fairness therefore holds only in a per-entity perpetual (§10).

Theorem 8.3 (Moral hazard elimination)**.** *The payout to deployer i is $V_i \cdot \lambda_i$. Since $\lambda_i \in [0, 1]$ and the layer enforces $Q_i \leq V_i$:*

$$\text{Payout}_i = V_i \cdot \lambda_i \leq V_i. \tag{20}$$

Proof. $V_i \lambda_i \leq V_i \cdot 1 = V_i$ since $\lambda_i \leq 1$. □

Theorem 8.4 (Coverage self-enforcement)**.** *The indicator $\mathbf{1}[\text{covered}]_i(t)$ is a deterministic function of $M_i(t)$ and $V_i/L_{\max}$ alone. No external governance is required for lapse or reinstatement.*

Proof. Immediate from (16): a deterministic function of $M_i(t)$ alone, updated continuously at every t. □

Theorem 8.5 (Leverage symmetry solvency)**.** *Conditional on $P(t)$ evolving continuously over every Δt_{liq} — guaranteed because the perpetual never settles instantaneously — under $L_{\max} = \lfloor 1/\Delta P^{\text{liq}}_{\max} \rfloor$, neither longs nor shorts can be driven to insolvency within Δt_{liq}.*

Proof. For notional Q with margin $M = Q/L_{\max}$: maximum loss over Δt_{liq} is $Q \Delta P^{\text{liq}}_{\max}$ for both sides. Solvency requires $Q \Delta P^{\text{liq}}_{\max} \leq Q/L_{\max}$, i.e. $\Delta P^{\text{liq}}_{\max} \leq 1/L_{\max}$. Since $L_{\max} \leq 1/\Delta P^{\text{liq}}_{\max}$ this holds. This bound applies to normal-state dynamics; it does not cover LP losses during an event-state oracle jump (§9.3). □

Theorem 8.6 (Payout sufficiency)**.** *Conditional on $f_{\max}$ being immutable after deployment (§9.1), the aggregate funding delivered over N^* intervals covers the full actual loss:*

$$\underbrace{V_{\text{snap}} \cdot N^* \cdot f_{\max}}_{\textit{perp delivers}} \geq \underbrace{V_i \cdot \lambda_i}_{\textit{layer pays out}} . \tag{21}$$

Proof. From $N^* = \lceil O(T^*)/f_{\max} \rceil \geq (V_i\lambda_i/V_{\text{snap}})/f_{\max}$. Multiplying by $V_{\text{snap}} f_{\max} > 0$ gives the result. □

Remark 8.7. The ceiling in N^* means the perp typically slightly overpays the layer. The excess $V_{\text{snap}} \cdot N^* \cdot f_{\max} - V_i\lambda_i \geq 0$ accumulates in the layer as a buffer.

Theorem 8.8 (Oracle calibration correctness)**.**

$$V_{\text{snap}} \cdot O(T^*) = V_i \cdot \lambda_i. \tag{22}$$

Proof. $V_{\text{snap}} \cdot (V_i\lambda_i/V_{\text{snap}}) = V_i\lambda_i$ by field cancellation. □

Theorem 8.9 (Singleton necessity)**.** *If two independent apportionment layers partition the pool into V^A_{snap} and V^B_{snap} and each calibrates using its own partial pool size, their combined payout is:*

$$V^A_{\text{snap}} \cdot O^A(T^*) + V^B_{\text{snap}} \cdot O^B(T^*) = 2V_i\lambda_i \neq V_i\lambda_i. \tag{23}$$

Proof. Each term equals $V_i\lambda_i$; the sum is $2V_i\lambda_i$. □

Theorem 8.10 (Pre-event snapshot stability)**.** *$O(T^*)$ is constant across all N^* intervals of the event window.*

Proof. $O(T^*)$ depends only on V_i (fixed at registration), λ_i (fixed from chain at processing time), and V_{snap} (frozen for the window by Definition 3.4). All three are constants during $[T^*, T^* + N^*\Delta t]$. □

9 Deployment Constraints and Security Requirements

The correctness results in §8 are conditional on the following deployment requirements, each supported by a constructive counterexample proving what breaks if the requirement fails.

9.1 $f_{\max}$ Immutability

Theorem 8.6 holds only if $f_{\max}$ is fixed after deployment. Reducing $f_{\max}$ post-commitment breaks payout sufficiency: with $V_i = V_{\text{snap}} \cdot f^{\text{original}}_{\max}$ and $\lambda_i = 1$, the perpetual delivers $V_{\text{snap}} \cdot f^{\text{reduced}}_{\max} < V_{\text{snap}} \cdot f^{\text{original}}_{\max} = V_i \cdot \lambda_i$.

Deployment requirement: $f_{\max}$ must be an immutable constructor parameter, not a settable field. Verifiable on-chain if the contract is non-upgradeable. Note that the apportionment layer does not need to store $f_{\max}$ internally — $N^* = \lceil O(T^*)/f_{\max} \rceil$ is computed externally from the oracle value $O(T^*)$ alone. The layer's payout obligation is fixed at $V_i \cdot \lambda_i$ regardless of $f_{\max}$; it is the *perp's* delivery guarantee that depends on $f_{\max}$ remaining unchanged.

9.2 Oracle Controller Security

A compromised oracle controller can extract any target amount T from LP shorts: the oracle value $O_{\text{malicious}} = T/V_{\text{snap}} + f_{\max}$ produces a payout window delivering $\geq T$.

Deployment requirement: the apportionment layer must be non-upgradeable, with no admin key holding oracle write access beyond the slashing detection logic. Contract immutability is on-chain verifiable; bug-freedom requires off-chain audit.

9.3 LP Window Persistence

Theorem 8.6 assumes LP shorts maintain positions for all N^* intervals. For large slashes in thin pools, LP short mark-to-market losses can exceed margin before the window closes, breaking payout delivery.

Deployment requirement: minimum pool size requirements relative to $V_{\max} = \max_i V_i$, and LP margin requirements calibrated to worst-case $O(T^*)$ rather than normal-state price dynamics. Not on-chain verifiable.

9.4 Lambda Observability (HIP-3 Specific)

In the HIP-3 instantiation, λ_i must be atomically readable from HyperCore at T^*. HIP-3 slashing is currently a validator committee decision, not an automatic on-chain state variable. The apportionment layer requires: (a) a read function for the slashing record on HyperCore, (b) a triggering mechanism, and (c) atomicity between reading λ_i and setting the oracle. This is the primary engineering gap between the mathematical framework and a deployable HIP-3 instantiation.

10 Cross-Subsidy, Adverse Selection, and LP Incentives

10.1 Cross-Subsidy in Pooled Coverage

With a single market-clearing $P(t)$, the pool prices at the capital-weighted average risk π_{pool} (Definition 3.4):

$$P(t) \approx \pi_{\text{pool}} = \frac{\sum_i \pi_i V_i}{V_{\text{snap}}}. \tag{24}$$

Each entity pays premium $V_i \cdot P(t)$, whereas actuarial fairness for entity i individually would require a rate of $V_i \cdot \pi_i$. The deviation is $(V_i \cdot \pi_i - V_i \cdot \pi_{\text{pool}})$ per unit time: positive (underpaying) when $\pi_i < \pi_{\text{pool}}$, negative (overpaying) when $\pi_i > \pi_{\text{pool}}$. The risk-adjusted weight w_i (Definition 3.5) measures each entity's intended burden share; the pooled market delivers this correctly only when $\pi_{\text{pool}} = \pi_i$ for all i — i.e. a homogeneous pool. Per-entity perpetuals would price each π_i exactly, eliminating the cross-subsidy at the cost of fragmented liquidity.

10.2 Adverse Selection Dynamics

The pool is stable if and only if every deployer's π_i lies within tolerance δ of the pool mark. When the spread between the highest and lowest π_i exceeds 2δ, some deployers are overcharged and will depart. The shift in pool mark when deployer i exits satisfies the accounting identity: the new mark is at least the old mark minus deployer i's contribution. This is an exact identity; it does not prove monotone worsening.

The death spiral is a qualitative economic argument: when low-risk deployers depart in response to overcharging, the remaining pool has higher average risk, raising the mark further and inducing further departures — a standard adverse selection dynamic.

HIP-3 implication. Auditor certification of π_i values is load-bearing for pool stability. Systematic miscertification drives pool composition toward higher risk over time, making the certification process a first-order operational concern.

Mitigations: per-entity risk tiers (subpools), minimum π_i certification thresholds for pool entry, dynamic premium adjustment.

10.3 LP Incentive Model

By Theorem 8.1, in actuarially fair equilibrium expected LP premium income equals expected payout exposure — zero net expected profit. Rational LP participation therefore requires one or more of:

(i) *Risk premium*: $P_{\text{mark}} > \pi_i$, endogenously determined by LP supply and demand.

(ii) *Collateral yield*: LP short margin earns yield while posted.

(iii) *Protocol incentives*: token rewards or fee rebates.

Which of these the HIP-3 instantiation relies on is an open design question.

11 Notation Summary

Object	**Expression / Note**
Risk parameter	$\pi_i = \mathbb{E}[\lambda_i \cdot \mathbf{1}[\mathcal{E}]] \in (0,1)$
Pool average	$\pi_{\text{pool}} = \sum_i \pi_i V_i / V_{\text{snap}}$
Premium weight	$w_i = V_i \pi_i / \sum_j V_j \pi_j$
Mark price	$P(t) \in (0,1)$, order-book determined
Normal oracle	$O(t) = 0$
Event oracle	$O(T^*) = V_i \lambda_i / V_{\text{snap}}$
V_{snap}	Pool total snapshotted at processing time
Event window	$N^* = \lceil O(T^*)/f_{\max} \rceil$ intervals
Funding rate	$f(t) = (P(t) - O(t))/\Delta t$
Normal funding	$P(t)/\Delta t > 0$, longs pay shorts
Event funding	$-f_{\max} < 0$, shorts pay longs
Required notional	$Q_i = V_i$
Moral hazard cap	$Q_i \leq V_i$ via $\min(\text{req}, V_i)$
Margin requirement	$M_i(t) \geq V_i / L_{\max}$
Max leverage	$L_{\max} = \lfloor 1/\Delta P^{\text{liq}}_{\max} \rfloor$, symmetric
Individual premium	$d\Pi_i = V_i \cdot P(t) \cdot \Delta t$ (normal); $V_i \cdot P(T^-) \cdot \Delta t$ (event window)
Perp aggregate payout	$V_{\text{snap}} \cdot N^* \cdot f_{\max} \geq V_i \lambda_i$
Deployer payout	$\text{Payout}_i = V_i \cdot \lambda_i$
Post-event price	$P(T^{*+}) = \gamma \cdot \pi^+_{\text{pool}}$, $\gamma > 1$, endogenous; qualitative
$f_{\max}$ immutability	Deployment constraint; §9.1
Oracle controller	Deployment security requirement; §9.2
LP window persistence	Payout sufficiency dependency; §9.3
Price equilibrium	Precondition for Theorem 8.1
LP participation	Requires risk premium or extrinsic benefit

Table 1: Notation summary. $\mathcal{E}$, V_i, λ_i are the only extrinsic mathematical inputs.

12 Conclusion

We have presented a two-layer framework implementing insurance as a perpetual derivative contract. The apportionment layer manages individual policies and routes payouts; the reinsurance perpetual provides aggregate capacity through LP shorts. The two oracle states deliver premium and payout through identical funding mechanics, with no reserve or claims process.

The eight main theorems establish self-consistency of the design: premiums are actuarially fair conditional on equilibrium; payouts match actual losses; coverage validity is self-enforcing; leverage caps protect both sides; the N^* ceiling construction guarantees payout delivery; oracle calibration is exact; the singleton constraint is a mathematical consequence of the calibration identity; and the event-window oracle is stable by construction.

The framework's correctness is conditional on the deployment requirements enumerated in §9. Of these, the most practically constraining for the HIP-3 instantiation are: (a) atomic readability of λ_i from HyperCore at T^*, (b) the immutability of $f_{\max}$, and (c) the security of the apportionment layer oracle controller. The mathematics guarantees correct behaviour given these conditions; it does not guarantee the conditions themselves. Audits of the deployed contract are a prerequisite for any production deployment.

References

[1] Lean 4 / Mathlib formalisation repository (three proof passes, 40 proved results, 10 axioms): `<link>`

[2] Apportionment layer testnet implementation (`ApportionmentLayer.sol`, oracle relay, Foundry deployment scripts, HyperEVM testnet): `<link>`

A Lean 4 Formalisation Record

All definitions and theorems in the main body have been machine-verified in Lean 4 / Mathlib (`leanprover/lean4:v4.28.0`) across three proof passes with zero `sorry`s and zero linter warnings. Only the standard axioms `propext`, `Classical.choice`, and `Quot.sound` appear in any proof. The consolidated repository is cited as **[1]**.

A.1 Proof Dependency Table

Result	Status	Depends on
Actuarial fairness uniqueness	Unconditional	—
Moral hazard elimination	Unconditional	—
Enforced notional cap	Unconditional	—
Coverage self-enforcement	Unconditional	—
Oracle calibration	Unconditional	—
Singleton necessity	Unconditional	—
Snapshot stability	Unconditional	—
Processing snapshot timing	Unconditional	—
$f_{\max}$ reduction breaks sufficiency	Unconditional	constructive witness
Oracle manipulation unbounded	Unconditional	constructive witness
LP liquidation risk exists	Unconditional	constructive witness
Pool stability / adverse selection	Unconditional	—
Leverage symmetry solvency	Conditional	price continuity (math axiom)
Payout sufficiency	Conditional	$f_{\max}$ immutability (deploy axiom)
Actuarial fairness	Conditional	price equilibrium (economic axiom)
LP participation	Structured axiom	risk premium or extrinsic benefit
Post-event dynamics (§7)	Qualitative	not formalised

Table 2: Proof dependency structure across three passes.

A.2 Axiom Register

Axioms fall into three categories.

Mathematical axioms. Require Mathlib infrastructure not yet fully developed.

- $\pi_i = \mathbb{E}[\lambda_i \cdot \mathbf{1}[\mathcal{E}]]$ — probability space / MeasureTheory.
- Continuity of $P(t)$ over Δt_{liq} — real analysis / Lipschitz continuity.
- $\lambda_i \in [0, 1]$, $V_{\text{snap}} > 0$, $f_{\max} > 0$ — parameter well-formedness.

Deployment security axioms.

- `fmax_immutability_required` — $f_{\max}$ is a constructor constant. On-chain verifiable if contract is non-upgradeable.
- `oracle_controller_is_secure` — apportionment layer has exclusive oracle write access. Immutability on-chain verifiable; bug-freedom requires audit.
- `lp_full_window_persistence` — LP shorts maintain positions for all N^* intervals. Not on-chain verifiable.

Economic / market microstructure axioms.

- `price_equilibrium_hypothesis` — $P(t) = \pi_i$ in equilibrium. Requires pool depth, informed participation, no arbitrage barriers; extrinsic to the on-chain mechanism. Carried as a named precondition in `actuarial_fairness_conditional`; the proof body does not yet formally derive fair pricing from equilibrium dynamics, which would require a stochastic market model beyond current Mathlib infrastructure.
- `lp_participation_exists` — structured axiom for rational LP participation; not proved.

A.3 Pass History and Superseded Theorems

Three proof passes were conducted. v3 imports and extends v2 without modifying any v2 definitions. Three v2 theorems were superseded:

v2 theorem	Issue	v3 replacement
`snapshot_stability`	Tautological (`rfl` on identical inputs)	`snapshot_immutable_during_window` via `SnapshotState`
`moral_hazard_control`	Passed hypothesis through unchanged	`enforced_notional_cap` derived from `min`
`actuarial_fairness`	Equilibrium implicit in hypothesis	`actuarial_fairness_conditional` with explicit axiom

Table 3: v2 theorems superseded in v3.